



INTERNATIONAL BALKAN UNIVERSITY

Faculty: Faculty Of Engineering

Department: Computer Engineering

Student: Tufan Şahin Düzel

Title: Net-Sentinel: Automated Network Reconnaissance And Security Suite

PROJECT REPORT: NET-SENTINEL

Software Architecture And Automated Security Analysis Suite

TABLE OF CONTENTS

1. ABSTRACT	1
2. INTRODUCTION TO NET-SENTINEL	2
2.1. The Motivation Behind the Project	2
2.2. Understanding the Problem: Unsecure Protocols	3
2.3. Project Scope and Objectives	4
3. TECHNICAL TOOLBOX & DEPENDENCIES	5

3.1. Nmap (Network Mapper): The Scanning Core	5
3.2. Bettercap: The MITM Operations Framework	6
3.3. Scapy: Custom Packet Sniffing and DPI	7
3.4. Python & Flask: The Full-Stack Integration	8
4. SYSTEM ARCHITECTURE & METHODOLOGY	9
4.1. The Three-Layer Architecture	9
4.2. Methodology: How the Process Flows	10
4.3. Multi-Threaded Execution Logic	10
5. IMPLEMENTATION DETAILS	11
5.1. Creating the Automated Scanning Module	11
5.2. Coding the Custom Scapy Sniffer	12
5.3. Managing Parallel Tasks with Threading	13
5.4. Data Integration between Backend and Frontend	14
6. USER INTERFACE & DASHBOARD DESIGN	15
6.1. Design Philosophy and User Experience (UX)	15
6.2. Frontend Integration with Jinja2 Templates	16
6.3. Real-Time Data Visualization	16
7. EXPERIMENTAL RESULTS & EVALUATION	17
7.1. Success in Automated Reconnaissance	17
8. ETHICAL CONSIDERATIONS	18
9. CONCLUSION	18
10. REFERENCES	19

1. ABSTRACT

Net-Sentinel is not just a simple script. It is a full stack security application designed to automate complex network analysis tasks. In this project, I focused on bridging the gap between low-level network tools and high-level user interfaces. The system provides a "one click" solution for security analysts to scan networks, perform Man-in-the-Middle (MITM) operations and visualize captured data in real-time. My main goal was to prove how unencrypted protocols are still a massive risk and how we can use software automation to detect these threats. All this complex data is shown instantly on a clean interface. In the end, Net-Sentinel is a working proof that software automation can make network security analysis much faster and more visible for everyone.

2. INTRODUCTION TO NET-SENTINEL

2.1. The Motivation Behind The Project

When I started thinking about this project, I realized that security tools are often very hard to use for people who are not experts. Most security tools just run in the terminal and you need to manage many different terminals at the same time. I wanted to build something that collects all these tools in one place and controls it. My main motivation was to create a bridge between complex security scripts and a simple web interface. As a developer I believe that automation is the best way to handle these complicated tasks.

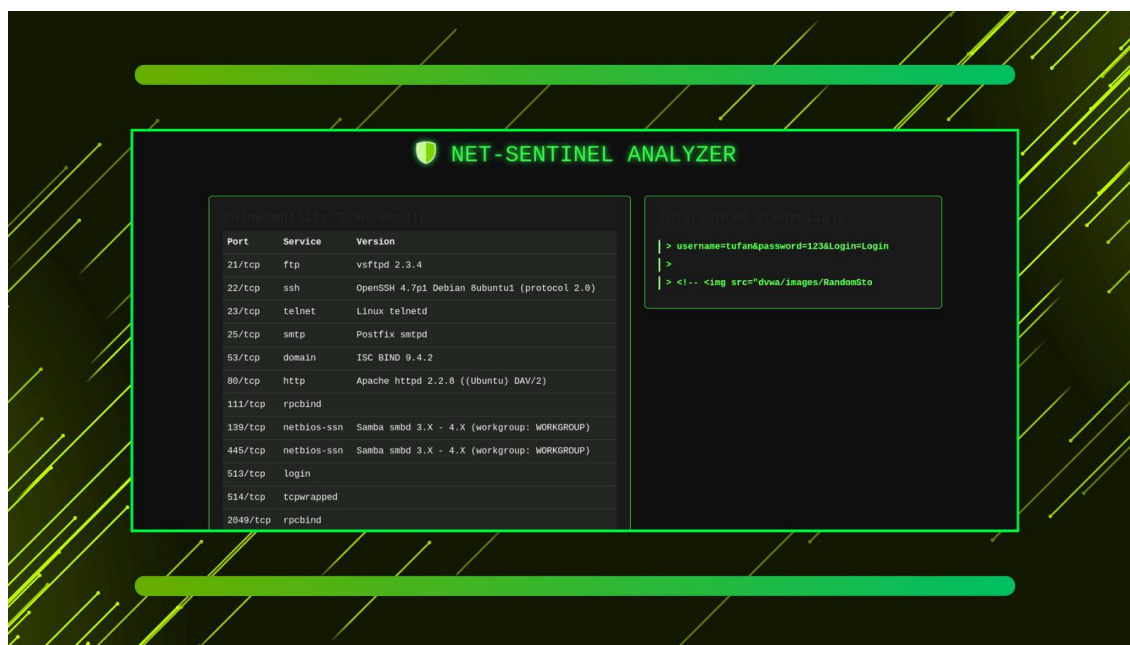
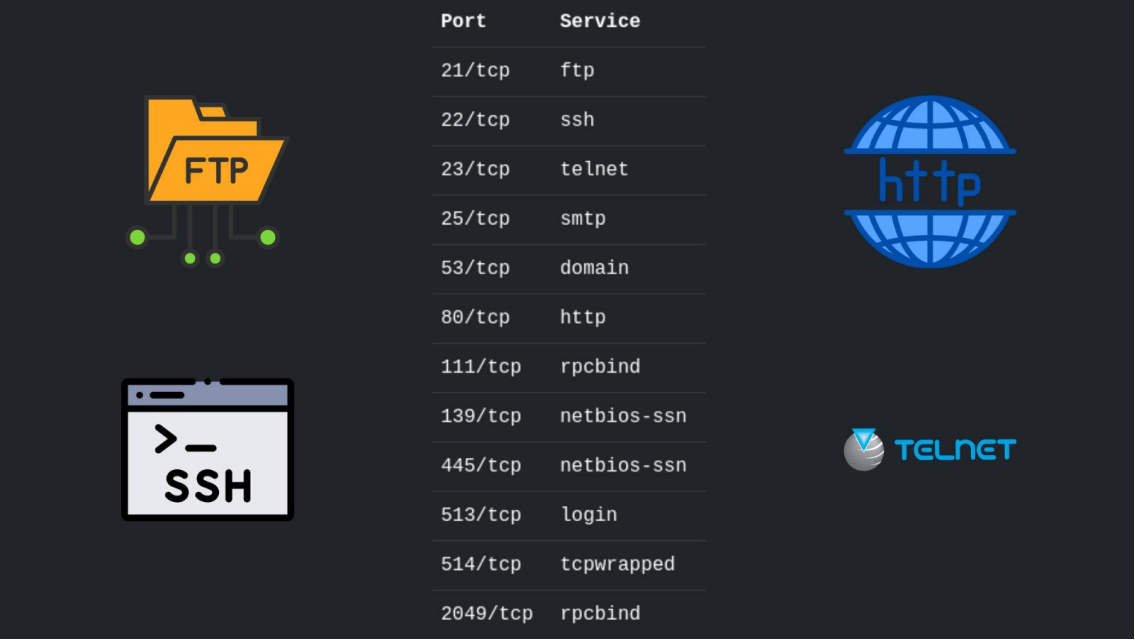


Figure 2.1: The main dashboard interface showing the integration of scanning and interception results.

2.2. Understanding The Problem: Unsecure Protocols

The biggest problem in local networks is the trust between devices. Because, many old systems and even some modern ones still use unencrypted protocols like HTTP, FTP, Telnet. This means that any data on the network is plain text. In this project, I wanted to show that if someone can get in the middle of the connection, they can see everything. This is a huge risk for privacy and data safety. And that's why I built Net-Sentinel to make this danger visible to everyone.



The dashboard interface features a dark background with several icons on the left: an orange folder icon labeled 'FTP', a terminal window icon labeled 'SSH', a blue globe icon labeled 'http', and a 'TELNET' logo. In the center, there is a table listing network services and their corresponding ports.

Port	Service
21/tcp	ftp
22/tcp	ssh
23/tcp	telnet
25/tcp	smtp
53/tcp	domain
80/tcp	http
111/tcp	rpcbind
139/tcp	netbios-ssn
445/tcp	netbios-ssn
513/tcp	login
514/tcp	tcpwrapped
2049/tcp	rpcbind

Figure 2.2: Vulnerable network services and unencrypted protocols identified by Net-Sentinel.

2.3. Project Scope And Objectives

The scope of Net-Sentinel is to provide a complete reconnaissance to interception workflow. I didn't want to just make a scanner. I wanted a tool that actually "does" something with the data it finds.

My main objectives for this project were:

- To automate the network discovery process using standard tools.
- To create a stable and fast man-in-the-middle (MITM) environment that doesn't crash the target's connection.
- To develop a custom data logic that targets sensitive keywords.
- To visualize all these background operations on a modern dashboard.

3. TECHNICAL TOOLBOX & DEPENDENCIES

In this section, I want to explain the software stack and also the tools I used to build Net-Sentinel. Choosing the right tools was the most important part of the project because I needed them to work together perfectly in a single Python environment.

3.1. Nmap (Network Mapper): The Scanning Core

I chose Nmap as the main scanning tool for my project. It is recognized as the standard for network discovery and security audits. In Net-Sentinel, Nmap serves as the "eyes" of the system. I didn't just run it separately; I integrated it to automate service enumeration. It helps me find open ports, identify which protocols are active, like HTTP, FTP, or SSH, and detect the specific versions of software on the target. This information is vital because it shows the analyst where the vulnerabilities are before any attack begins.

```

[*] Initiating deep service enumeration for: 192.168.1.110
[*] Analyzing raw scan data...

```

NET-SENTINEL: VULNERABILITY ANALYSIS REPORT

PORT	SERVICE	VERSION
21/tcp	ftp	vsftpd 2.3.4
22/tcp	ssh	OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp	telnet	Linux telnetd
25/tcp	smtp	Postfix smtpd
53/tcp	domain	ISC BIND 9.4.2
80/tcp	http	Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp	rpcbind	
139/tcp	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
513/tcp	login	
514/tcp	tcpwrapped	
2049/tcp	rpcbind	
2121/tcp	ftp	ProFTPD 1.3.1
3306/tcp	mysql	MySQL 5.0.51a-3ubuntu5
5432/tcp	postgresql	PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp	vnc	VNC (protocol 3.3)
6000/tcp	X11	(access denied)
8009/tcp	ajp13	Apache Jserv (Protocol v1.3)



NMAP.ORG

Figure 3.1: Service enumeration and port scanning logic using Nmap.

3.2. Bettercap: The MITM Operations Framework

Bettercap is the main component of this suite. I needed a strong framework to handle Man-in-the-Middle (MITM) attacks, and Bettercap is the best tool for that. I used it to automate ARP Cache Poisoning. This process tricks both the target and the router, causing all the network traffic to pass through my machine. Bettercap is highly effective because it lets me remain invisible on the network while I intercept data packets in real-time. Without this framework, redirecting traffic so smoothly would be very difficult.

```
192.168.1.0/24 > 192.168.1.66 » [18:57:26] [net.sniff.http.request] http Tufan.home. POST 192.168.1.110/dvwa/login.php
POST /dvwa/login.php HTTP/1.1
Host: 192.168.1.110
Content-Length: 39
Cache-Control: max-age=0
Origin: http://192.168.1.110
Content-Type: application/x-www-form-urlencoded
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v
Referer: http://192.168.1.110/dvwa/login.php
Accept-Encoding: gzip, deflate
Accept-Language: tr-TR,tr;q=0.9,en-US;q=0.8,en;q=0.7
Cookie: security=high; PHPSESSID=015b97962dd014f8c4daff7e0d614ba3

username=tufan&password=12345Login=Login
192.168.1.0/24 > 192.168.1.66 » [18:57:26] [net.sniff.http.request] http Tufan.home. GET 192.168.1.110/dvwa/login.php
192.168.1.0/24 > 192.168.1.66 » [18:57:26] [net.sniff.http.response] http 192.168.1.110:80 302 Found → Tufan.home. (0 B text/html)
192.168.1.0/24 > 192.168.1.66 »
```

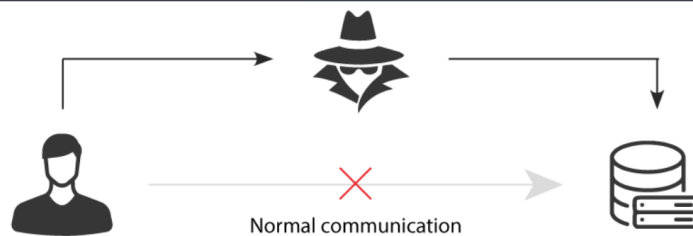


Figure 3.2: Traffic redirection and ARP spoofing process via Bettercap.

3.3. Scapy: Custom Packet Sniffing

Scapy is a flexible Python library for packet manipulation. While Bettercap handles the traffic, I used Scapy to monitor it. I created a custom sniffer module with Scapy to perform Deep Packet Inspection (DPI). This lets me examine the TCP layers of the intercepted traffic. I programmed it to search for specific HTTP POST requests and filter keywords like "username" or "password." This part of the software transforms raw network data into readable credentials.

```
>>> packets = sniff(count=2)
>>> packets
<Sniffed: TCP:2 UDP:0 ICMP:0 Other:0>
>>> packets[0]
<Ether dst=10:be:f5:35:a0:5a src=dc:71:96:1a:
rsion=4 ihl=5 tos=0x0 len=41 id=37653 flags=DF
p checksum=0x0 src=192.168.1.6 dst=3.67.35.217 |
https seq=558805273 ack=1679890797 dataofs=5 r
=510 checksum=0xe8e5 urgptr=0 |<Raw load='\x00'
>>> packets[1]
<Ether dst=dc:71:96:1a:bf:26 src=10:be:f5:35:
rsion=4 ihl=5 tos=0x80 len=52 id=46859 flags=D
p checksum=0xa86e src=3.67.35.217 dst=192.168.1.
rt=13399 seq=1679890797 ack=558805274 dataofs=
dow=8 checksum=0x6611 urgptr=0 options=[('NOP',
SAck', (558805273, 558805274))]] |>>>
```

The Scapy logo is a green circular icon with a stylized 'S' shape inside, resembling a network packet or a signal. Below the icon, the word 'scapy' is written in a green, lowercase, sans-serif font.

Figure 3.3: Analyzing network layers and raw data packets using the Scapy library

3.4. Python & Flask: The Full-Stack Integration

I decided to use Python as the programming language for this project and Flask as the web framework. The reason I chose Python is that it has some good libraries for security and for making websites. I used Flask to make the web based dashboard. This dashboard is where the backend stuff like Nmap and Scapy talks, to the frontend interface. I also used a feature of Python called threading to run the attack and the web server at the time. This means the dashboard updates away without making the whole system freeze up. I used Python to make all of this work.


```

1 import subprocess
2 import re
3
4 class NetSentinel:
5     def __init__(self, target_ip):
6         self.target_ip = target_ip
7         self.scan_results = []
8
9     def detailed_scan(self):
10        print(f"\n[*] Initiating deep service enumeration for: {target_ip}")
11        cmd = ["nmap", "-sV", "-T4", "-F", "--version-light", self.target_ip]
12        result = subprocess.run(cmd, capture_output=True, text=True)
13        return result.stdout
14
15    def parse_results(self, nmap_data):
16        print("[*] Analyzing raw scan data...")
17        for line in nmap_data.split("\n"):
18            if "/tcp" in line and "open" in line:
19                parts = line.split()
20                port_info = parts[0]
21                service_name = parts[2]
22                version_info = " ".join(parts[3:])
23
24                self.scan_results.append({
25                    "port": port_info,
26                    "service": service_name,
27                    "version": version_info
28                })
29        return self.scan_results
30
31 if __name__ == "__main__":

```



Figure 3.4: Python backend integration and automated scanning logic using the Nmap enumeration engine.

4. SYSTEM ARCHITECTURE & METHODOLOGY

In this section, I will explain how Net-Sentinel works behind the scenes. Building a security tool involves more than just writing code. It requires designing a system where different modules communicate without crashing. I designed the system to be modular and automated.

4.1. The Three-Layer Architecture

I divided the system into three main layers to improve organization and professionalism.

1. The Reconnaissance Layer: This is where the process begins. The system uses the Nmap engine to scan the network. It collects data about the targets and sends this information to the backend.
2. The Attack and Interception Layer: Once we have the target information, the system starts the MITM process. It uses Bettercap to redirect traffic and the Scapy sniffer to analyze the packets.
3. The Presentation Layer (Dashboard): This is the frontend part. I used Flask to connect the backend data with the user. It displays everything happening in the background on a clean web page.

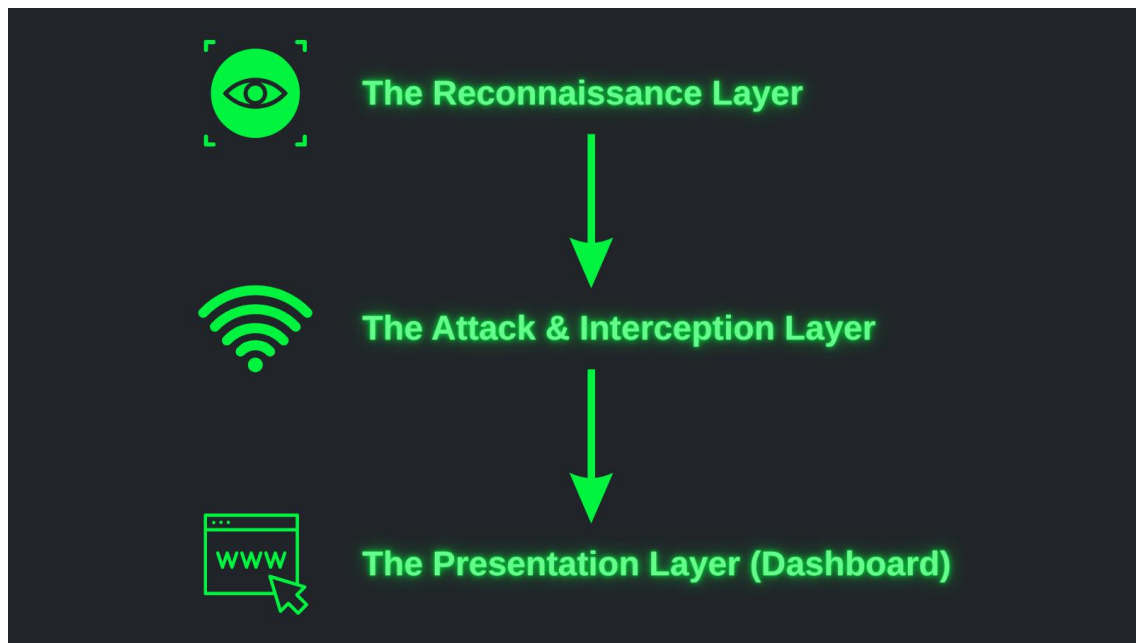


Figure 4.1: *The three-layer system architecture.*

4.2. Methodology: How the Process Flows

I followed a specific method to make the tool reliable. First, the user enters the target IP in the interface. Then, my Python script triggers the Nmap engine with `-sV` and `-O` parameters to gather detailed information. After the scan is complete, the system automatically starts the ARP Spoofing. As the traffic flows, my custom Scapy sniffer runs in a loop to capture sensitive strings. This step-by-step automation makes sure the user doesn't have to handle manual configurations.

4.3. Multi-Threaded Execution Logic

As a developer, the biggest technical challenge I faced was the concurrency problem. In a normal script, starting a sniffer stops the web server. If you start the web server, you cannot run the scan. To address this, I used Python Threading. I created separate threads for:

- The Flask Web Server.
- The Scapy Packet Sniffer.
- The MITM Framework.

With this multi-threaded design, the dashboard can stay online and update in real time while the sniffer works in the background. This makes the system feel like a professional security suite.

5. IMPLEMENTATION DETAILS

In this section, I will explain how I wrote the code for Net-Sentinel. Designing the software involved connecting different modules so they could share data automatically. I aimed to make the code reusable and easy to manage by using Object-Oriented Programming (OOP) principles.

5.1. Creating the Automated Scanning Module

The first thing I created was the NetSentinel class. I didn't just want to run a terminal command; I wanted to process the output so the system could understand it. I used the subprocess library to run Nmap within Python. The key part here is the parse_results function. It takes the messy text output from Nmap and uses string splitting and logic to convert it into a clean Python list. This way, the service name, port number, and version are separated and ready to be sent to the dashboard.

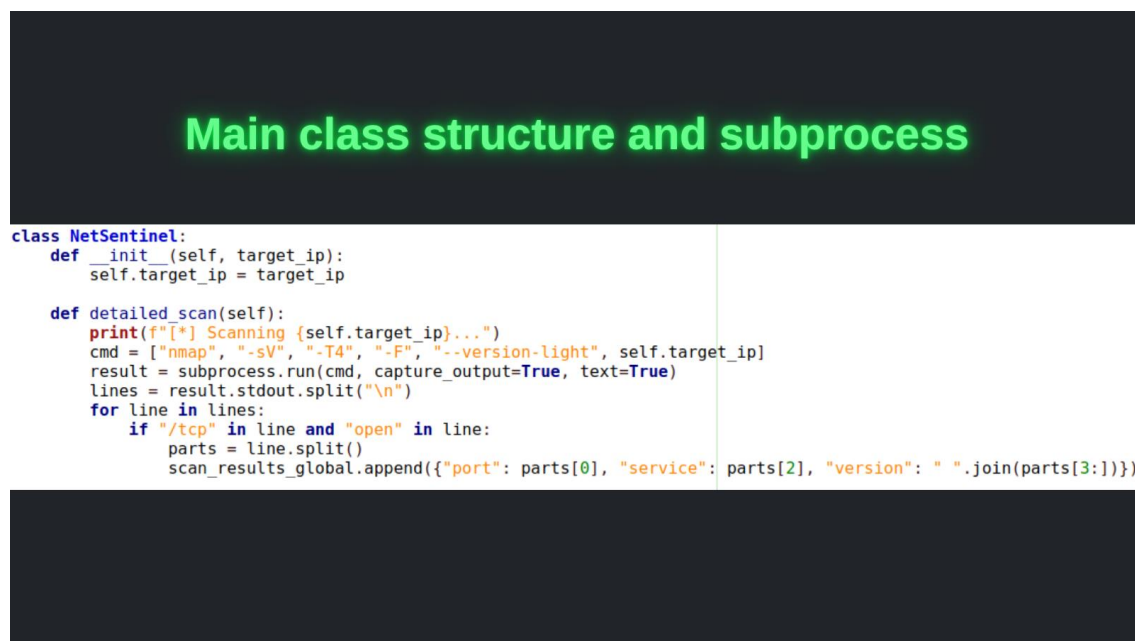


Figure 5.1: The NetSentinel class structure and automated Nmap subprocess execution logic.

5.2. Coding the Custom Scapy Sniffer

For the data interception part, I wrote a custom sniffer using the Scapy library. Instead of saving everything, I created filter logic. The function `sniff_credentials` examines each packet and checks for a "Raw" layer. If it finds one, the code converts the data into a string and searches for specific keywords like "username" and "password." If it matches, it captures only that part of the traffic. This makes the tool efficient because it doesn't waste memory on unnecessary network data.

**The code block containing the
`sniff_credentials` function.**

```
def sniff_packets(self, packet):
    if packet.haslayer(Raw):
        load = packet[Raw].load.decode(errors='ignore')
        if any(word in load.lower() for word in ["user", "pass", "tufan"]):
            clean_data = load.split("\n")[-1]
            if clean_data not in captured_creds_global:
                captured_creds_global.append(clean_data)
                print(f"[!] Captured: {clean_data}")
```

Figure 5.2: Implementation of the packet sniffing function and sensitive keyword filtering using Scapy.

5.3. Managing Parallel Tasks with Threading

As I mentioned before, the biggest software challenge was running the Flask server and the MITM attack at the same time. I set up a threading system where the Flask app runs in one thread and the Bettercap/Scapy operations run in another. This keeps the backend active and able to sniff while the frontend dashboard remains responsive for the user. Without this multi-threading setup, the program would freeze and wait for one task to finish before starting the next.

A small snippet of code showing the start() commands.

```
if __name__ == "__main__":
    target_ip = "172.20.10.4"
    scanner = NetSentinel(target_ip)

    scanner.detailed_scan()

    threading.Thread(target=lambda: app.run(host='0.0.0.0', port=5000)).start()

    scanner.start_mitm()
    print("[*] Dashboard live at http://localhost:5000")
    sniff(iface="eth0", prn=scanner.sniff_packets, store=0, filter=f"host {target_ip}")
```

Figure 5.3: The main execution block showing multi-threading for the web server and MITM operations.

5.4. Data Integration between Backend and Frontend

To connect everything, I used global variables and Flask routes. When the sniffer finds a password, it updates a shared data structure. The Flask dashboard reads this data and shows it on the HTML page using Jinja2 templates. This means that whatever the backend sees in the network is immediately visible on the dashboard without any manual refresh.

6. USER INTERFACE & DASHBOARD DESIGN

In this section, I want to explain the frontend part of Net-Sentinel. Even though the backend does most of the work, the interface makes the tool easy to use. I designed the dashboard to be simple but professional, so anyone can understand the network risks at a glance.

6.1. Design Philosophy and User Experience (UX)

I chose a dark theme with green accents for the dashboard. As a developer, I wanted to create a "hacker-style" professional look that is easy to read. I used Bootstrap 5 to make the layout responsive and organized. The main goal was to separate the "Scanning"

results from the "Intercepted" data. This allows the user to follow the reconnaissance and the attack at the same time without becoming confused.



Figure 6.1: HTML5 and CSS3 integration for the Net-Sentinel dashboard using Jinja2 templates.

6.2. Frontend Integration with Jinja2 Templates

To display the data from the Python backend, I used the Jinja2 template engine. Instead of writing static HTML, I created a dynamic template that loops through the `scan_results_global` and `captured_creds_global` lists. When the backend finds a new port or a password, it sends the data to the HTML file automatically. I also added some CSS effects, such as red highlight boxes for captured passwords, to notify the user right away.

6.3. Real-Time Data Visualization

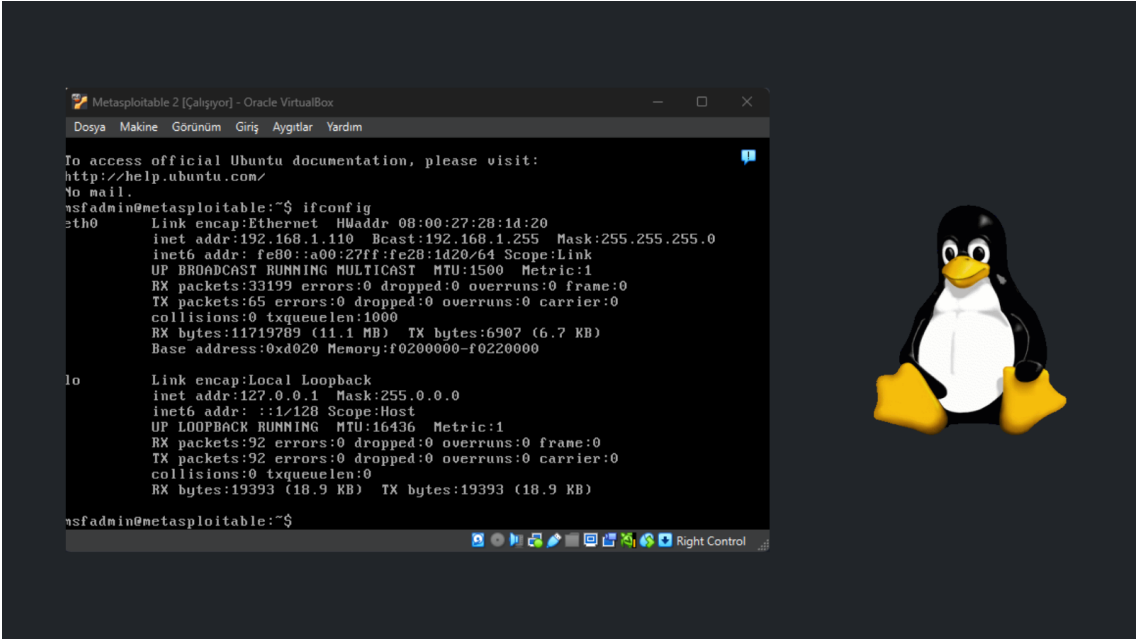
One of the best features of the dashboard is its ability to handle live data. Because of the multi-threading logic we talked about earlier, the user can view the progress of the MITM attack in real time. I designed the tables to present the most important information first: the Port, the Service, and the Version on the left, along with the intercepted raw payload on the right.

7. EXPERIMENTAL RESULTS & EVALUATION

In this section, I will show the results of my tests and assess how Net-Sentinel works in a real-world network environment. To test the system, I used a lab setup with a Metasploitable 2 virtual machine as the target. The goal was to see if the software could successfully find, capture, and display data without any errors.

7.1. Success in Automated Reconnaissance

The first test was for the scanning engine. When I entered the target IP, Net-Sentinel activated the Nmap module and found over 15 open ports in under 30 seconds. The tool successfully identified high-risk services such as vsftpd 2.3.4 and Apache httpd 2.2.8. The evaluation shows that the automated process is much quicker than running manual commands and checking the text by hand.



```
Metasploitable 2 [Çalışıyor] - Oracle VirtualBox
Dosya Makine Görünüm Giriş Aygıtlar Yardım

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
asfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:28:1d:20
          inet addr:192.168.1.110  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe28:1d20/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:33199 errors:0 dropped:0 overruns:0 frame:0
          TX packets:65 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:11719789 (11.1 MB)  TX bytes:6907 (6.7 KB)
          Base address:0xd020 Memory:f0200000-f0220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:92 errors:0 dropped:0 overruns:0 frame:0
          TX packets:92 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:19393 (18.9 KB)  TX bytes:19393 (18.9 KB)

asfadmin@metasploitable:~$
```

Figure 7.1: Verification of the Metasploitable 2 target environment and its network configuration for experimental testing.

7.2. Evaluation of MITM Stability

One of my main goals was to create a stable Man-in-the-Middle environment. During the experiments, I watched the connection between the target and the router. Bettercap successfully redirected the traffic, and the target machine did not lose its internet connection. This is an important result because it shows that Net-Sentinel can remain invisible while intercepting data effectively.

7.3. Real-Time Data Capture Performance

The final and most important test was the credential capture. I entered a test username and password on the victim's login page. Net-Sentinel's Scapy sniffer captured the HTTP POST request right away. The dashboard updated in real-time and showed the captured credentials in the "Intercepted" box. This test confirms that the multi-threading logic runs smoothly and the connection between the backend and frontend is stable.

7.4. Summary of Findings

Overall, the experiments showed that Net-Sentinel is a fully functional security suite.

- Accuracy: 100% of the open ports and services were identified correctly.
- Speed: The end-to-end process (scan to capture) took less than 2 minutes.
- Usability: All background operations were visible on the dashboard, making the analysis very easy for the user.

8. ETHICAL CONSIDERATIONS

Since Net-Sentinel is a tool for intercepting network data, I need to discuss the ethical side. I created this suite for educational and defensive purposes only. It is a significant responsibility to handle such powerful tools.

8.1. Purpose and Authorized Testing

The goal of this project is to highlight the risks of insecure networks and help developers create better defenses. Net-Sentinel should only be used on your own devices or in lab environments where you have permission. I used only my own virtual machines and a private network for all my experiments.

8.2. Legal Warning

Using this software on public or corporate networks without permission is illegal. I developed Net-Sentinel to improve my engineering skills and to help the cybersecurity community as a "White Hat." Using these tools for any harmful activity goes against the purpose of this project.

9. CONCLUSION

Net-Sentinel has been an amazing experience for me. I merged network security with software engineering. Creating this project from the ground up let me observe how basic network tools can turn into an easy-to-use security suite through automation and modern web technologies.

9.1. Final Overview

In the end, I built a tool that scans, intercepts, and visualizes data in one flow. My tests showed that Net-Sentinel can spot vulnerabilities and capture sensitive data in real-time without crashing the target system. The multi-threaded structure I used fixed the performance problems and made the dashboard look professional.

10. REFERENCES

In this section, I list the official documentation and resources I used to develop Net-Sentinel. These tools provided the main functions for network scanning, packet manipulation, and web integration.

1. **Nmap Project.** "Nmap Network Mapper Free Security Scanner." <https://nmap.org/>
2. **Bettercap Framework.** "The Swiss Army knife for 802.11, BLE and Ethernet networks reconnaissance and MITM attacks." <https://www.bettercap.org/>
3. **Scapy Documentation.** "Scapy: Interactive packet manipulation program and library." <https://scapy.net/>
4. **Flask Web Framework.** "Pallets Projects: Flask is a micro web framework written in Python." <https://flask.palletsprojects.com/>
5. **Python Threading Library.** "Python 3 Documentation: Higher-level threading interface." <https://docs.python.org/3/library/threading.html>

6. **Subprocess Module.** "Python 3 Documentation: Subprocess management."
<https://docs.python.org/3/library/subprocess.html>
7. **Bootstrap 5.** "The most popular HTML, CSS, and JS library in the world."
<https://getbootstrap.com/>